NeurIPS 2024 · PRINCETON UNIVERSITY

SWE-agent

Agent-Computer Interfaces Enable Automated Software Engineering

Yang, Jimenez, Wettig, Lieret, Yao, Narasimhan, Press

Presented by

Khalil Dabbah · Riad Bkheet

MAY 5, 2026

Where we're going

01	**The Problem**	What real software engineering looks like for an LLM
02	**The Solution — ACI**	A new kind of interface, designed for agents not humans
03	**How It Works**	Search, edit, view — and the 4 design principles behind them
04	**Results & Evidence**	12.5% on SWE-bench, ablations, what each piece contributes
05	**Critique & 2026 Reality**	What aged well, what didn't, what survived
06	**Activity**	Kahoot quiz on what you just learned

First, what is SWE-bench?

Before we talk about the agent, you need to know the test it was built for.

2,294	12	1
real GitHub issues	popular Python repos	task: produce a patch that passes all hidden unit tests

HOW EACH TASK WORKS

1	**Issue described in plain English**	*"Function returns wrong type when input is empty"*
2	**Whole repo is given to the agent**	*Hundreds of files, thousands of lines*
3	**Agent must produce a patch (diff)**	*Like a real GitHub PR*
4	**Hidden unit tests are run on the patch**	*Pass = task resolved. Fail = nothing.*

Can an LLM fix a real bug in a real codebase?

Imagine giving GPT-4 a Python repo with thousands of lines and saying: "There's a bug — find it and fix it."

1
FIND
Locate the right file

2
READ
Understand the code

3
EDIT
Modify it correctly

4
TEST
Verify the fix works

THE CHALLENGE BEFORE 2024

The standard answer was: give the LLM a Linux shell.

Same tools humans use — bash, grep, sed, vim. The result: only 3.8% of bugs fixed (best non-interactive baseline).

LLMs aren't humans. The tools we use don't fit them.

Too many flags	grep alone has 30+ options. The LLM gets confused or picks the wrong ones.
No feedback after edits	sed runs silently. The LLM doesn't know if the edit worked or what the file looks like now.
Search dumps too much	grep -r returns hundreds of lines. The context window fills with noise.
No guardrails	Broken syntax gets written to disk. Errors cascade across multiple turns.

Humans got VSCode for this. LLM agents need their own VSCode.

Why don't humans code in raw bash either?

We use VSCode, IntelliJ, PyCharm — interfaces designed around human strengths and weaknesses.

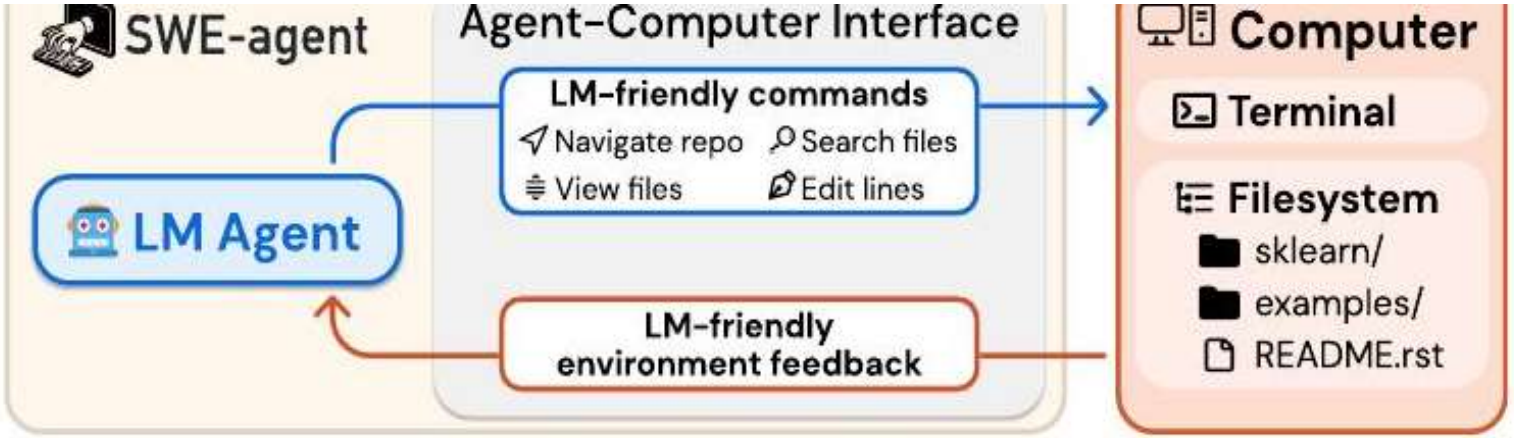
HUMANS	LLM AGENTS NEED...
✓ Syntax highlighting	Concise feedback (no context bloat)
✓ Autocomplete	Few simple commands
✓ Visual file tree	Auto-shown file state after edits
✓ Inline error messages	Rejection of broken syntax
✓ Click-to-navigate	Compact, multi-line operations

The paper's claim: build the second column. Call it the Agent-Computer Interface.

ACI

Agent-Computer Interface

A custom layer between the LLM and the computer — built for what LLMs need.



LM-friendly commands

Few options, simple syntax

Structured feedback

Show file state after every edit

Built-in guardrails

Reject syntactically broken edits

How to build a good ACI

1 Simple actions

Few commands, few options. SWE-agent gives only ~7 commands: open, goto, scroll_up, scroll_down, find_file, search_dir, edit. Each takes 1-3 args.

2 Compact actions

One command should do a meaningful chunk of work. The edit command replaces a multi-line block in a single call — not 5 separate sed steps.

3 Informative feedback

Show the result clearly, but not too much. After every edit, the file viewer auto-shows the updated section. No silent operations.

4 Guardrails

Catch mistakes early. A syntax linter inspects every edit before saving — if there's an indentation error, the edit is rejected, not committed.

How does the agent FIND code?

Three approaches the authors compared. Result: it really matters.

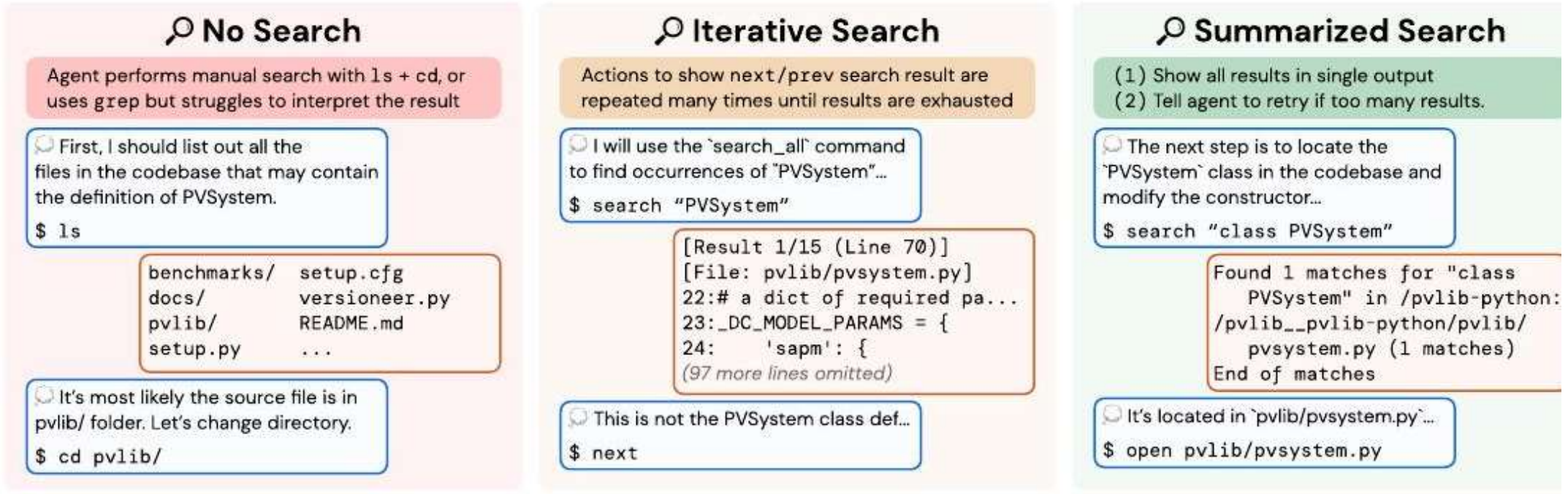


Figure 5: Three different Search interfaces for task instance pvlib pvlib-python-122

Figure 5 — from the paper

No Search — 15.7% **Iterative — 12.0% ↓** **Summarized — 18.0% ✓**

Sometimes more options = worse performance.

ITERATIVE SEARCH (12.0%)

Like Vim's :next — show one match at a time

Behavior:

Agent calls 'next, next, next, next…' until exhausted. Doesn't stop when it finds the right thing.

Why it fails:

LLMs are trained to be thorough. With 50 results, the agent walks through ALL of them and burns its context window.

Lesson:

What works for humans ("show me one at a time") doesn't work for LLMs.

SUMMARIZED SEARCH (18.0%)

All results in one block, capped at 50

Behavior:

Agent sees "Found 3 matches: file_a.py, file_b.py, file_c.py" — picks one and moves on.

Why it works:

Compact view → one decision. If 50+ results, the system tells the agent: refine your query.

Lesson:

Compact > granular. Decisions > exploration.

How does the agent CHANGE code?

The most critical piece. Removing it drops performance by 7.7 points.

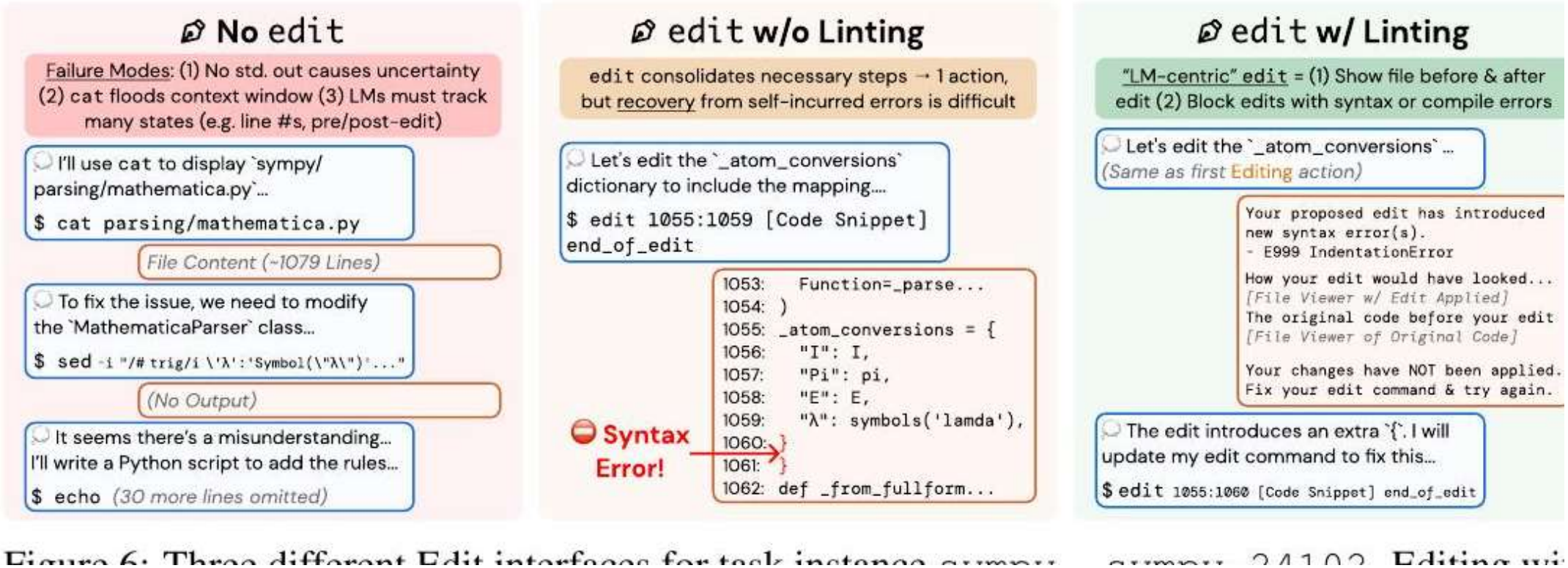


Figure 6: Three different Edit interfaces for task instance sympy... sympy-24102. Editing wi...

Figure 6 — from the paper

No edit (bash only) — 10.3%

edit w/o linting — 15.0%

edit w/ linting — 18.0% ✓

Same task, two interfaces

The agent wants to fix lines 404-407 in a Python file.

WITH BASH

Multiple commands, no feedback

```
$ cat -n file.py | head -420
$ sed -i '404,407s/.../.../' file.py
(no output)
$ cat -n file.py | sed -n '400,410p'
...syntax broken, not detected
```

✗ No feedback after the edit
✗ Syntax errors slip through
✗ Multi-step, error-prone

WITH ACI

One command, structured feedback

```
> edit 404 407 <new code>
[Linter check] ✓ valid syntax
[File: solvers/diophantine.py]
402:    diop_type = 'cubic_thue'
404:    elif (total_degree > 3 and
405:    len(set(k.exp[k.is_Pow]))
```

✓ File auto-shown after edit
✓ Linter rejects bad syntax
✓ One command, one decision

The hidden third pillar: managing what the LLM sees.

LLMs have a context window. Fill it with junk → performance drops sharply.

Collapse old observations

Only the last 5 outputs are shown in full. Earlier ones get squashed into 1 line. Avoids stale file content polluting context.

Last 5 obs: 18.0% · Full history: 15.0%

Drop old error messages

When the agent makes a malformed command, it gets an error. Once recovered, all error messages except the first are silently dropped.

Keeps the chat clean and focused

Consistent output template

Even "no output" gets a message: 'Your command ran successfully and did not produce any output' — so the LLM never has to guess what happened.

Removes ambiguity

Ablation result: full chat history hurts performance by 3 percentage points.

From 3.8% to 12.5% — a 6.7× improvement

On SWE-bench: 2,294 real GitHub issues from 12 popular Python repos.

12.5%
SWE-bench full
(SWE-agent w/ GPT-4 Turbo)

18.0%
SWE-bench Lite
(canonical 300 instances)

3.8%
Previous state-of-the-art
(retrieval-augmented baseline)

$1.67
Avg. cost per resolved
instance (in 2024 prices)

Paper's Table 1 — main results

set. We benchmark models in the SWE-agent, Basic CLI, and Retrieval Augmented Gen
(RAG) settings established in SWE-bench [20].

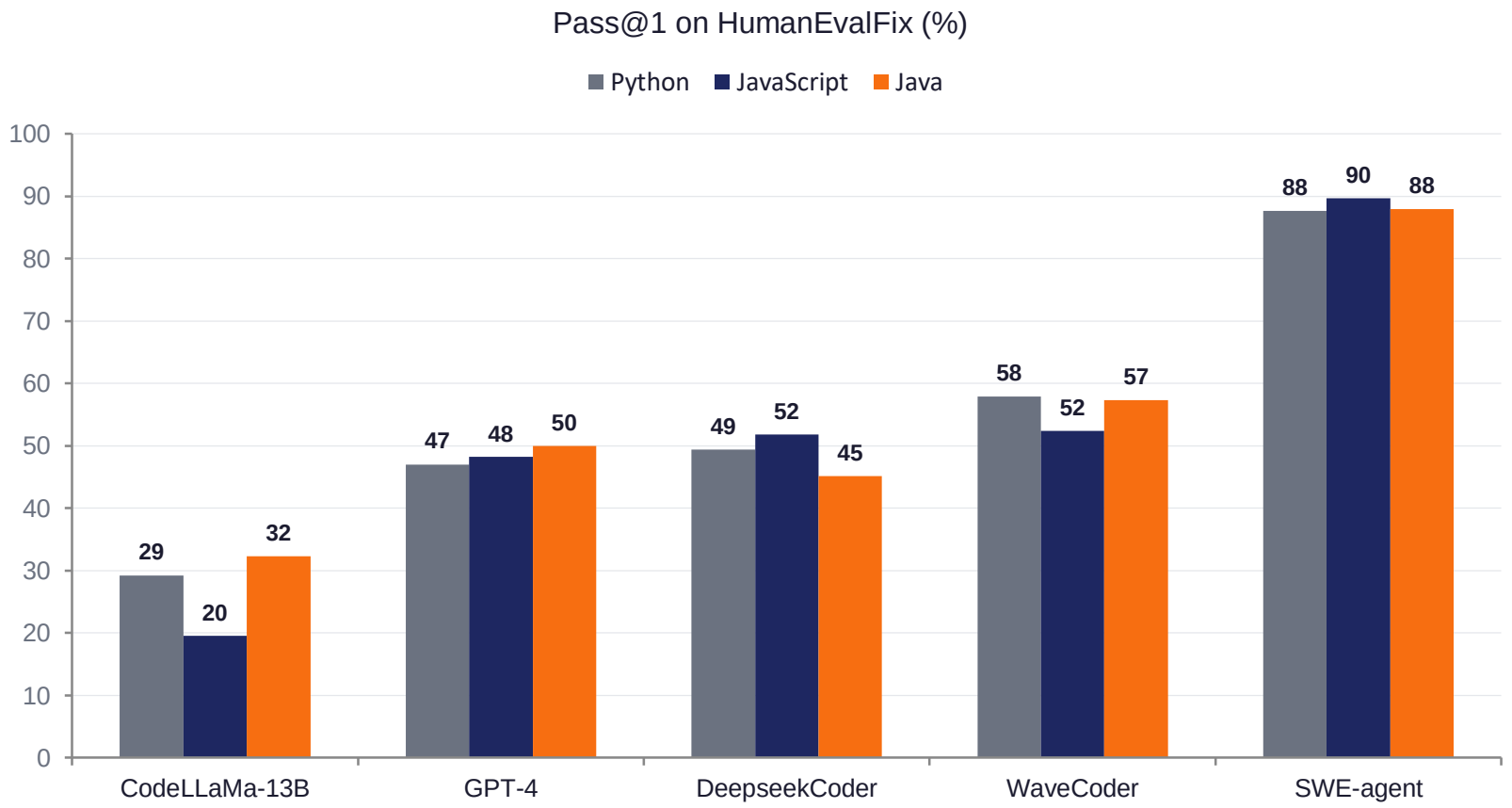
Model	SWE-bench % Resolved	SWE-bench $ Avg. Cost	SWE-bench Lite % Resolved	SWE-bench Lite $ Avg. Cost
RAG				
w/ GPT-4 Turbo	1.31	0.13	2.67	0.13
w/ Claude 3 Opus	3.79	0.25	4.33	0.25
Shell-only agent				
w/ GPT-4 Turbo	-	-	11.00	1.46
w/o Demonstration	-	-	7.33	0.79
SWE-agent				
w/ GPT-4 Turbo	**12.47**	1.59	**18.00**	1.67
w/ Claude 3 Opus	10.46	2.59	13.00	2.18

Table 2: Pass@1 results on HumanEvalFix [32]. Except for
SWE-agent, we use scores as reported in Yu et al. [65].

Model	Python	JS	Java

It also works on simpler bug-fixing tasks

HumanEvalFix: short-form code debugging across 3 languages — Python, JavaScript, Java.



WHAT THIS MEANS

87.7%

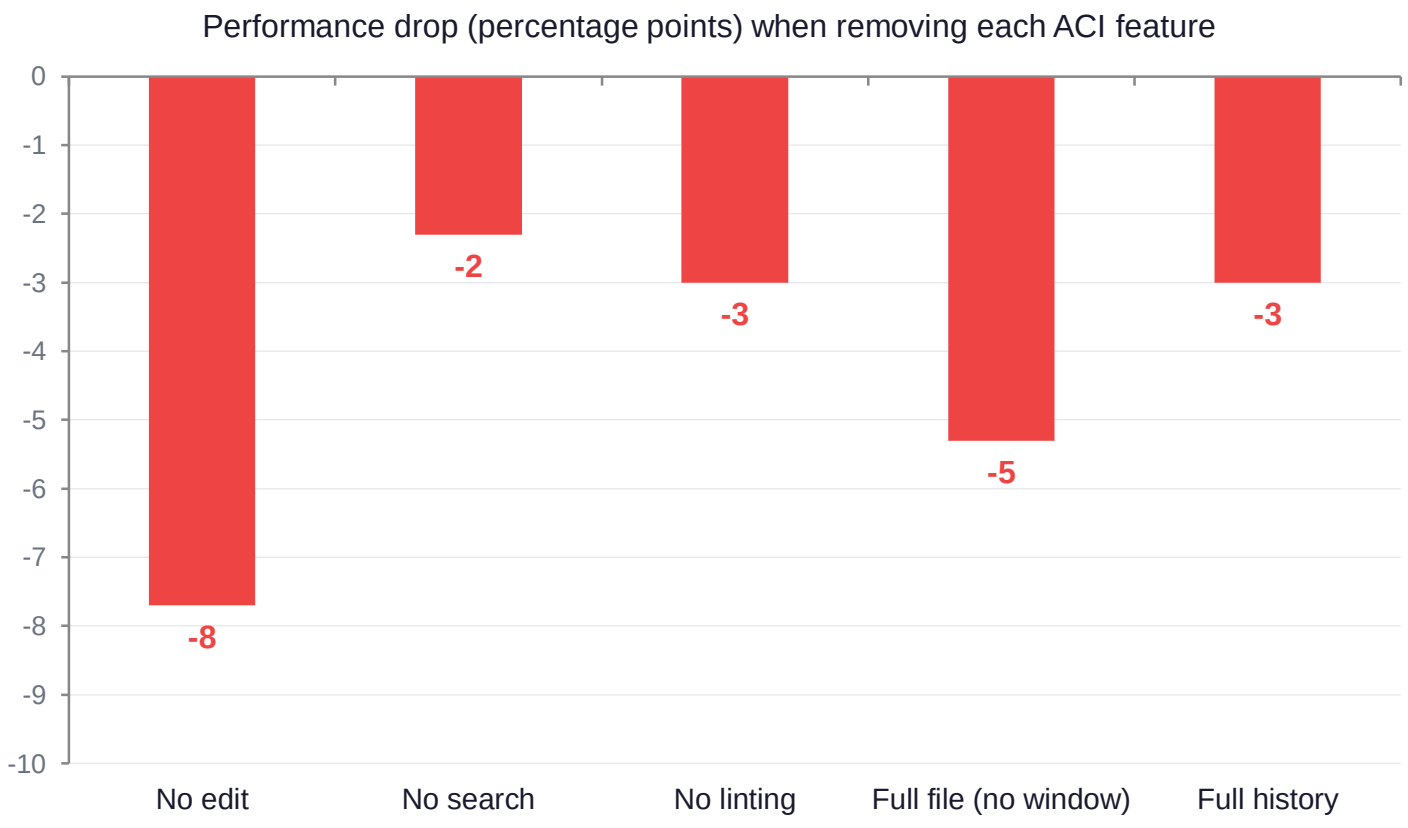
On simpler tasks, SWE-agent nearly saturates the benchmark.

+30 points over GPT-4 alone (no ACI).

Generalizes across 3 languages — not Python-only.

Turn off any part of the ACI — performance drops.

Strongest evidence in the paper. Each design choice provably helps.



KEY TAKEAWAYS

Edit interface is the most critical
Removing it drops performance by 7.7 pts

Window size matters
Full file (no scrolling) hurts more than 30 lines

Linting is a real win
Without it, syntax errors compound

Context management isn't optional
Full chat history actively hurts

Even the authors say SWE-agent has weaknesses.

Read between the lines of the paper — they flag these issues themselves.

Some bash commands still leak through

SWE-agent is built ON TOP of bash. Agents can still run cd, ls, grep, cat — and they often do, exhausting their budget on inefficient exploration.

ACI tuned via grid search on the test set

Window size, history length, and other parameters were chosen by sweeping over SWE-bench Lite. Authors say: "We arrived at the final ACI design through qualitative analysis on hand-picked examples."

Tested on only 2 (closed-source) LLMs

GPT-4 Turbo and Claude 3 Opus. Smaller models (Llama 3, DeepSeek Coder) performed "subpar" — the ACI may need very strong base models to work at all.

$4 budget cap per task

Many runs hit the budget limit and were submitted automatically. Without budget constraints, performance might be different — but more expensive.

Stop and look at the models for a second...

These models are over 2 years old. The world has changed dramatically.

GPT-4 Turbo

2-YR OLD

`gpt-4-1106-preview`

Released: Nov 2023

Context window

128K tokens

SWE-bench score

12.47%

Cost per fix

$1.59

Claude 3 Opus

2-YR OLD

`claude-3-opus-20240229`

Released: Feb 2024

Context window

200K tokens

SWE-bench score

10.46%

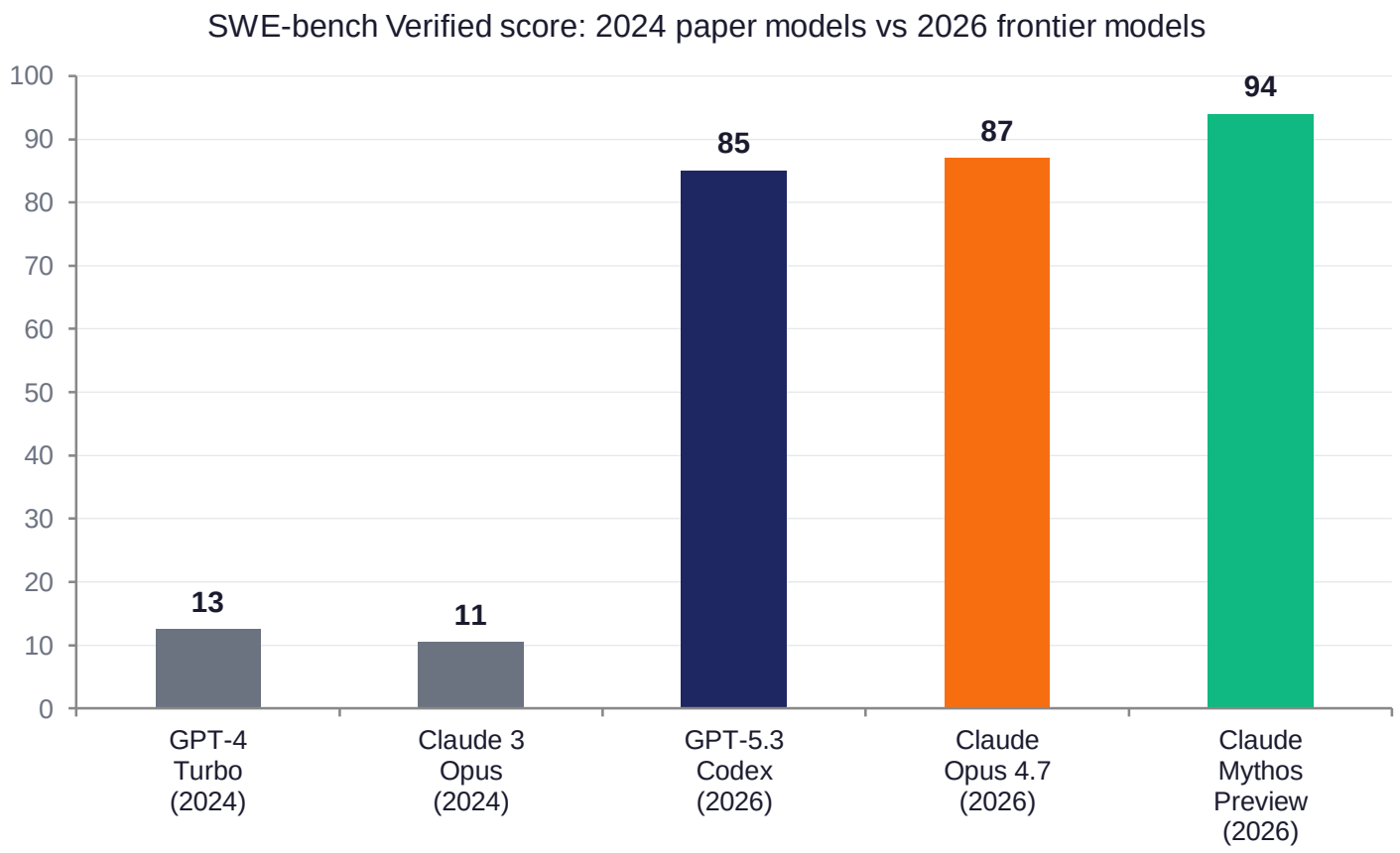
Cost per fix

$2.59

Next slide: what these models look like in 2026.

Same benchmark, vastly stronger models

Two years of progress: 5–7× context window, 6–8× higher score on the same test.



2026 FRONTIER MODELS

Claude Opus 4.7

1M context · 87% SWE-bench

GPT-5.3 Codex

1M context · 85.0% SWE-bench

Claude Mythos Preview

1M context · 93.9% SWE-bench

The model improved more than the interface ever did.

The benchmark used to validate SWE-agent is now considered unreliable.

In February 2026, OpenAI announced it was abandoning SWE-bench Verified as a measure of frontier coding ability — citing widespread training-data contamination.

Training leakage

GitHub issues + their fix PRs sit in Common Crawl and The Stack. Models trained after mid-2024 likely saw the solutions.

Memorization, not reasoning

Studies showed top models can identify the file path of a fix from the issue ID alone (76% accuracy) — without seeing the code.

Score inflation

Models that score 80%+ on Verified score 45–58% on the contamination-resistant SWE-bench Pro. Same model, half the score.

So: 12.5% on a flawed test is NOT proof of capability.

The ACI was tuned on the test it was scored on.

Classic software engineering red flag — your evaluation should not drive your design.

WHAT THEY DID

Tuned hyperparameters via grid search

On SWE-bench Lite — the same benchmark they report results on.

Window size: 100 lines

Why 100? Because grid search said so on this dataset.

History: keep last 5 observations

Why 5? Same reason — fit to this benchmark.

Linter targeted at Python

SWE-bench is Python-only.

OPEN QUESTIONS

Does it work on TypeScript?

The Python linter wouldn't apply. Window size may need to change.

Does it scale to 1M-line codebases?

SWE-bench repos are small to medium. Real enterprise codebases are 100× larger.

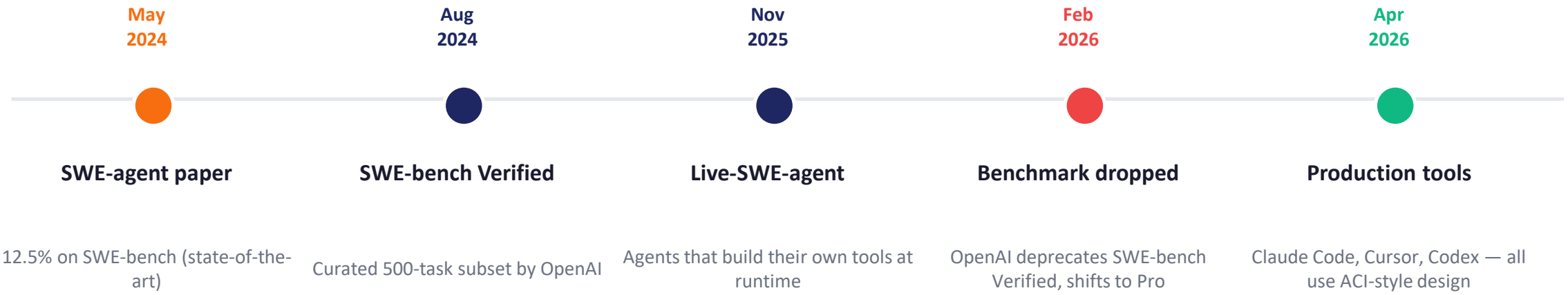
Does it work for non-coding agents?

Database admin? DevOps? Security pentest? Paper doesn't say.

What's the methodology to design ANY ACI?

"Manual inspection + grid search" isn't really a methodology.

In 18 months, SWE-agent went from SOTA to reference implementation.



WHAT SURVIVED?

The ACI idea — not the system.

Modern coding agents (Claude Code, Cursor, Codex) all ship with custom interfaces — file viewers, structured edits, syntax-aware feedback. The principles are now standard. The specific implementation in the paper is now mostly historical.

The ACI idea now lives in production tools.

Each one took the ACI principles and pushed them further.

Claude Code

87.6%

on SWE-bench Verified

Terminal-based agent. CLAUDE.md project memory, MCP integration, file viewers, structured edits. The most direct descendant.

Cursor

67.2%

on SWE-bench Verified

IDE-anchored. Combines inline autocomplete, agent mode, and parallel cloud agents. Visual + agentic.

OpenAI Codex

71.0%

on SWE-bench Verified

Multi-surface: CLI + IDE + cloud delegation. Background agents that submit PRs autonomously.

Live-SWE-agent

45.8%

on SWE-bench Verified

Most radical: lets the agent write its own tools at runtime. ACI is no longer fixed — it evolves per task.

KEY MESSAGE

The real contribution is the idea — not the system.

LLM agents are a new class of users. They need interfaces designed for them — not the same tools we built for humans.

I

The principle (ACI) outlives the implementation.

II

The numbers age with the benchmark, not with the idea.

III

Every coding agent today owes something to this paper.